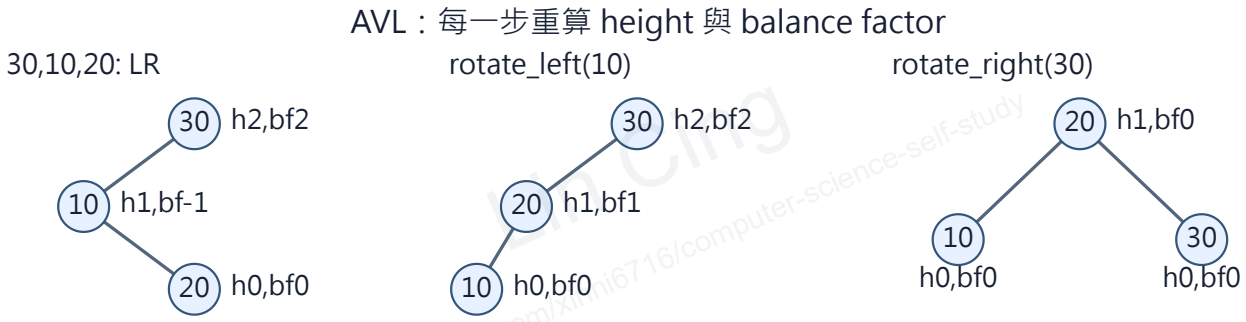
第 10 章推導解答

DS10-01 • Optimal BST 的區間 DP

單鍵 C11=0.1、C22=0.2、C33=0.7。區間 1..2 總重 0.3，選根 1 成本 0.3+0.2=0.5，根 2 成本 0.3+0.1=0.4，故 root=2。區間 2..3 總重 0.9，根 2 成本 1.6，根 3 成本 1.1，故 root=3。全區總重 1，候選根 1、2、3 的成本依序 2.1、1.8、1.4，故根 3，其左子為 2，再左子為 1。含失敗機率時，若 dummy leaf 也計一次比較，就用空區間 e=q；若只計真鍵比較，所有失敗查詢各少一次，需一致調整成本與遞迴，不可把兩套初始化與結論混用。

DS10-02 • AVL 插入與刪除差異

插入 30 後高 0，插 10 後 30.bf=1、高 1。插 20 後 10.bf=−1，30.bf=2，屬 LR，先左旋 10，再右旋 30，得到根 20、左右 10、30，全樹高 1。刪除例可取根 4，左子 2 具有子 1、3，右子 5 是葉。原根 bf=1。刪 5 後根 bf=2，左子 bf=0，應單右旋，得到根 2、左 1、右 4 且 4 左 3。較高子 bf=0 是刪除常見特例，不應誤判為雙旋。修復後是否繼續向上由子樹高度變化判定。



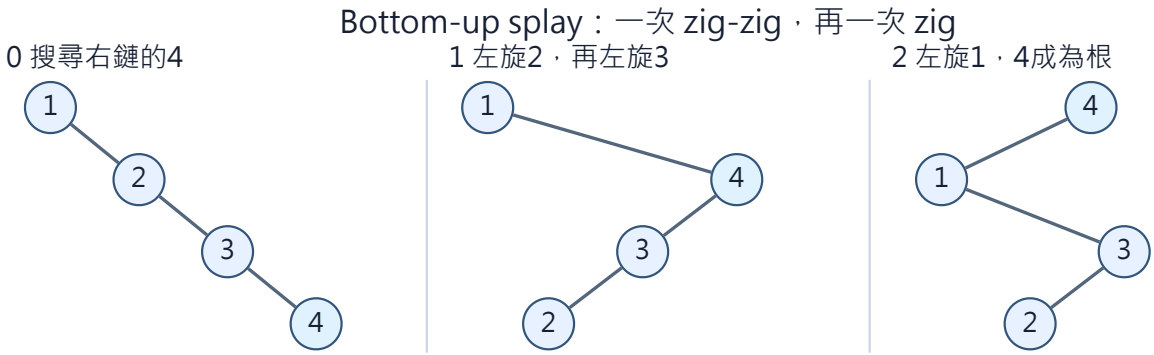
DS10-03 • 紅黑樹的黑高與插入

10 初設黑。5 紅接左，不違規。1 紅接 5 左，父紅、叔 NIL 黑，對 10 右旋並換色，得根 5 黑、左右 1 紅、10 紅。插 7 成 10 左紅，父 10 紅且叔 1 紅，將兩者染黑、祖父 5 染紅，再把根 5 染黑。最終 5 黑，左 1 黑，右 10 黑、其左 7 紅。每條根到 NIL 都含根 5、下一黑點 1 或 10、及 NIL，共 3 黑。此例紅連結只有 10→7，恰左傾，但一般紅黑合法性不推出所有操作結果都左傾。例如黑根的紅右子在一般紅黑樹合法，LLRB 不接受。

DS10-04 • Splay 的攤銷界

x=4、父 3、祖父 2 同為右側，先左旋 2，再左旋 3，局部根成 4，其左子 3、3 左子 2；整體仍掛在 1 右邊。接著 4 的父為根 1，做一次左 zig，最終根 4，左子 1，1 右子 3，3 左子 2。此單次實際搜尋可走 Θ(n) 鏈。攤銷是 `Σc_i=Σĉ_i+Φ_initial−Φ_final`，不是每個 c_i 都小。任意起始樹 Φ_initial 可為 O(n log n)，因此任意 m 次操作可給 O(m log n+n log n) 的通用初始勢能界；從空樹開始則初始項可省，並計入建樹操作。

第 10 章推導解答 （續）



DS10-05 • 紅黑刪除修復的局部證明

本題另以「含子樹根」計每條路徑黑點數，令 x 側實際為 b，缺額補回後為 b+1。兄弟 w 子樹實際為 b+1，w 本身黑，故近侄與遠侄子樹各為 b。遠侄原紅，改黑後整個遠侄子樹成 b+1。旋轉後 p 為黑，左右是 x 與原近侄，兩者各 b，故 p 子樹也成 b+1。w 升為根並取得 p 原色 c，c=0 表紅、1 表黑，整個局部對外黑數為 c+b+1，恰等於刪除前的 p 子樹。若 p 原紅，w 升後紅，其兩子 p 與遠侄已黑，沒有 red-red；若 p 原黑，w 亦黑。缺額在局部內消除，不需上推。這個局部計數慣例已明訂，不能直接把 b 與投影片排除根的 bh(x) 混用。

DS10-06 • 後繼移植、顏色與元素身分

z是節點20，兩子皆非NIL；後繼y是節點25，原色紅，x=y.right=NIL。y.parent是30而非z，因此先transplant(25,NIL)，令30.left=NIL且當前NIL.parent=30。再令25.right=30、30.parent=25，transplant(20,25)令root=25、25.parent=NIL；接25.left=10、10.parent=25，25.color取20原黑，釋放20。

最後黑根25，左10黑，右30黑，30右40紅。removed_color取25在舊位置的紅色，所以不呼叫delete_fix；把25接到根時染黑只是替代原黑根20，不是額外拿走一個黑。所有路徑為25B→10B→NIL、25B→30B→NIL或25B→30B→40R→NIL，各含3黑（含根與NIL）。中序從10,20,25,30,40變10,25,30,40，恰少20。

節點25只是換位置，指向元素25的handle仍指同一活物件。複製25內容到原20節點再刪原25，雖可得到相同鍵集合，卻令原25的handle失效、原20的handle看見25，違反所要求的身分契約。若誤用z原黑而進入缺額修復，則對沒有少黑的位置進行修復，其前提已不成立。

第 10 章推導解答 （續）

DS10-07 • 詳解：相同目標不代表相同旋轉

(1) 右旋 30，再右旋 20。按節點 [10,20,30] 記錄子樹大小，三態為 [3,5,7]、[3,7,3]、[7,5,3]。四個葉的 rank 為 0，故前後 $\Phi=\log_2 105\approx 6.714$，$\Delta\Phi=0$，攤銷成本為 2。最後樹為 T(10,5,T(20,15,T(30,25,35)))。
(2) 左旋 10，再右旋 30。按 [20,10,30] 記錄 s，三態為 [3,5,7]、[5,3,7]、[7,3,3]。前 $\Phi=\log_2 105$，後 $\Phi=\log_2 63\approx 5.977$，$\Delta\Phi=\log_2(3/5)\approx -0.737$，攤銷成本約 $1.263\le 2\log_2(7/3)\approx 2.445$。最後為 T(20,T(10,5,15),T(30,25,35))。
(3) 第一例的舊 x 子樹是 {5,10,15}，新 g 子樹是 {25,30,35}；第二例的兩個新子樹分別是 {5,10,15}、{25,30,35}，兩例皆不重疊。反向做第一例，右旋 20 後再右旋 30，會得到 T(10,5,T(30,T(20,15,25),35))，不是標準 zig-zig 結果。此時舊 x 子樹與新 g 子樹在節點 15 重疊，無法使用原本的互斥集合論證；應說「原證明不適用」，不能只憑這一例便推斷所有序列的複雜度。

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.
Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study